

Lecture 2: Information - Sources and Representation

EC 441 - Introduction to Computer Networking

Boston University, Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

1. Define information quantitatively using Shannon entropy
2. Calculate the entropy of simple probability distributions
3. Distinguish between data and information
4. Convert between different units (bits, bytes, SI, binary prefixes)
5. Explain how text is encoded (ASCII, UTF-8, base64)
6. Analyze real files to understand representation efficiency
7. Understand basic binary file formats and magic numbers
8. Estimate data rates for multimedia (audio, images, video)

1 What is Information?

1.1 Intuitive Definition

Information is what reduces uncertainty. Consider these scenarios:

- “The coin landed on heads” - provides information if you didn’t know the outcome
- “Water is wet” - provides little information (you already knew this)
- “The network packet arrived” vs “The network packet was lost” - depends on context

1.2 Shannon’s Information Theory (1948)

Key insight: The amount of information in a message is related to how *surprising* it is.[1]

Surprisal (self-information) of an event with probability p :

$$I(p) = -\log_2(p) \text{ bits} \tag{1}$$

Examples:

- Coin flip ($p = 0.5$): $I = -\log_2(0.5) = 1$ bit
- Rolling a 6 on a die ($p = 1/6$): $I = -\log_2(1/6) \approx 2.58$ bits
- Certain event ($p = 1$): $I = -\log_2(1) = 0$ bits (no surprise!)

1.3 Entropy: Average Information

For a random variable X with outcomes $\{x_1, x_2, \dots, x_n\}$ and probabilities $\{p_1, p_2, \dots, p_n\}$:

$$H(X) = - \sum_{i=1}^n p_i \log_2(p_i) \text{ bits} \quad (2)$$

Entropy $H(X)$ represents:

- The average amount of information per message
- The minimum number of bits needed to represent the outcome
- The fundamental limit on data compression

1.4 Example Calculation

Fair coin flip:

- Two outcomes: H, T
- Each has $p = 0.5$
- $H(X) = -[0.5 \log_2(0.5) + 0.5 \log_2(0.5)]$
- $H(X) = -[0.5(-1) + 0.5(-1)] = 1 \text{ bit}$

Biased coin ($p(H) = 0.9$, $p(T) = 0.1$):

- $H(X) = -[0.9 \log_2(0.9) + 0.1 \log_2(0.1)]$
- $H(X) = -[0.9(-0.152) + 0.1(-3.322)]$
- $H(X) \approx 0.469 \text{ bits}$

Key insight: Less randomness = less information needed to represent outcomes!

1.5 Visualizing Entropy: $H(p)$

For a Bernoulli random variable (binary outcome with probability p), the entropy function has an elegant shape:

Key observations:

- Maximum entropy at $p = 0.5$: most uncertain outcome
- Entropy = 0 at $p = 0$ or $p = 1$: completely certain outcome
- Symmetric around $p = 0.5$
- At $p = 0.1$ or $p = 0.9$: $H(p) \approx 0.47 \text{ bits}$ (less than half the maximum)

2 Units of Information

2.1 Basic Units

Note: In networking, we often use “octet” to avoid confusion (byte historically meant different things on different systems).

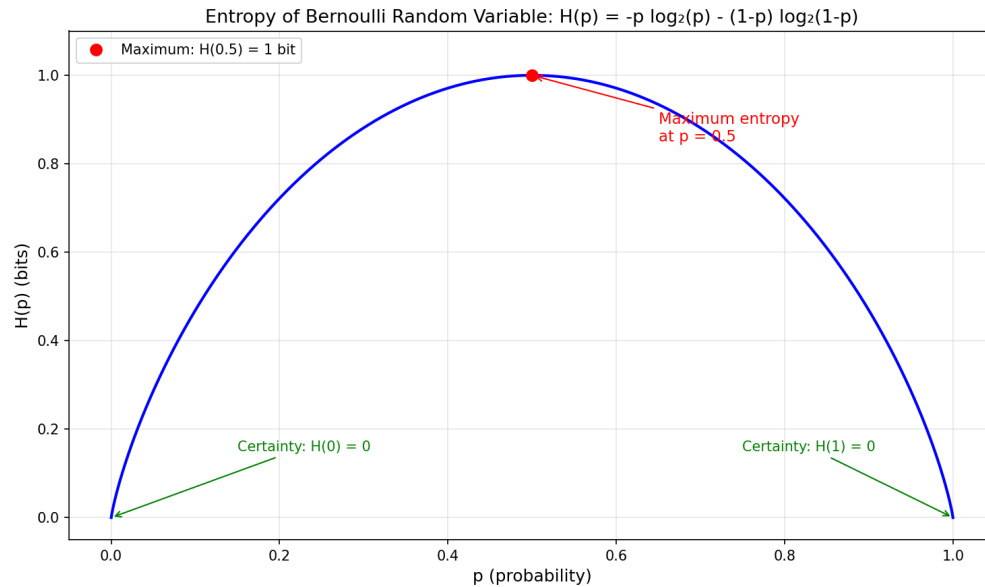


Figure 1: Entropy $H(p)$ reaches maximum (1 bit) at $p = 0.5$ (maximum uncertainty) and approaches 0 as p approaches 0 or 1 (certainty). See `plot_entropy.py` for the code that generated this figure.

Unit	Value	Symbol
Bit	Binary digit (0 or 1)	b
Byte	8 bits	B
Octet	8 bits (networking term)	-
Nibble	4 bits	-

2.2 SI Prefixes (Decimal, base-10)

Prefix	Symbol	Value	In bits/bytes
Kilo	k	10^3	1,000
Mega	M	10^6	1,000,000
Giga	G	10^9	1,000,000,000
Tera	T	10^{12}	1,000,000,000,000

Used for: Network speeds (100 Mb/s = 100,000,000 bits/sec), storage device marketing

2.3 Binary Prefixes (base-2, IEC standard)[2]

Used for: RAM sizes, file sizes in OS, actual storage capacity

2.4 Common Confusion

A “1 TB” hard drive:

- Marketing: 1,000,000,000,000 bytes (SI)
- OS shows: ~ 931 GiB (binary)

Prefix	Symbol	Value	In bytes
Kibi	Ki	2^{10}	1,024
Mebi	Mi	2^{20}	1,048,576
Gibi	Gi	2^{30}	1,073,741,824
Tebi	Ti	2^{40}	1,099,511,627,776

- Difference: $(10^{12}/2^{30}) \approx 931$

3 Data vs. Information

3.1 Key Distinction

- **Information** = the actual content/meaning (measured in entropy)
- **Data** = how we represent that information (measured in bits/bytes)

Fundamental relationship:

$$\text{Data size} \geq \text{Information content} \quad (3)$$

The difference represents:

- Redundancy (for error correction/detection)
- Inefficient encoding
- Metadata/overhead

3.2 Example: Storing English Text

English text has ~ 1.5 bits of entropy per character (due to letter frequencies, context).

Representations:

1. **ASCII**: 8 bits/char (efficiency: $1.5/8 = 18.75\%$)
2. **Huffman coding**: ~ 4.7 bits/char (efficiency: $1.5/4.7 \approx 32\%$)
3. **Arithmetic coding**: ~ 1.7 bits/char (efficiency: $1.5/1.7 \approx 88\%$)
4. **Optimal (theoretical)**: 1.5 bits/char (100%)

3.3 Tradeoffs

Representation	Efficiency	Accessibility	Use Case
Plain ASCII	Low	High	Human-readable files
Compressed	High	Low	Storage, transmission
Error-coded	Low	Medium	Noisy channels

In networking: We constantly balance these tradeoffs!

4 Text Encoding

4.1 ASCII (American Standard Code for Information Interchange)[3]

- **Size:** 7 bits per character (128 characters)
- **Extended ASCII:** 8 bits (256 characters)
- **Range:** 0x00 - 0x7F (standard ASCII)

Examples:

```
'A' = 0x41 = 65 = 0b01000001
'a' = 0x61 = 97 = 0b01100001
'0' = 0x30 = 48 = 0b00110000
' ' = 0x20 = 32 = 0b00100000
'\n' = 0x0A = 10 = 0b00001010
```

Key ranges:

- Control characters: 0x00-0x1F
- Digits: 0x30-0x39 ('0'-'9')
- Uppercase: 0x41-0x5A ('A'-'Z')
- Lowercase: 0x61-0x7A ('a'-'z')

4.2 Unicode and UTF-8[4]

Problem: ASCII only handles English. What about , , ?

Unicode: Universal character set with 149,186+ characters (and growing)

- Each character has a **code point**: U+0041 (A), U+4F60 ()

UTF-8 Encoding: Variable-length encoding

- 1 byte: ASCII characters (U+0000 to U+007F)
- 2 bytes: Most Latin, Greek, Cyrillic, Arabic, Hebrew
- 3 bytes: Most Asian characters, emoji
- 4 bytes: Rare characters, some emoji

Example:

```
"Hello" -> 5 bytes (all ASCII)
"Hello " -> 5 + 1 + 6 = 12 bytes
H e l l o
41 65 6C 6C 6F 20 E4BDA0 E5A5BD
```

Why UTF-8 dominates the web:

- Backward compatible with ASCII
- Space-efficient for English text
- Self-synchronizing (can find character boundaries)
- No byte-order issues

4.3 UTF-16

- **Size:** 2 or 4 bytes per character
- **Used by:** Windows, Java, JavaScript internally
- **Pros:** Simpler indexing for many Asian languages
- **Cons:** Wastes space for ASCII/Latin text, byte-order issues (BOM)

4.4 Base64 Encoding[5]

Purpose: Represent binary data using only ASCII printable characters

Alphabet: A-Z, a-z, 0-9, +, / (64 characters = 6 bits)

Encoding process:

1. Take 3 bytes (24 bits) of input
2. Split into 4 groups of 6 bits
3. Map each to a base64 character
4. Add '=' padding if needed

Example:

```
Input:  "Man" = 0x4D 0x61 0x6E
Binary: 01001101 01100001 01101110
Groups: 010011 010110 000101 101110
Base64: T      W      F      u
Result: "TWFu"
```

Efficiency: 3 bytes → 4 characters (33% overhead)

Use cases:

- Email attachments (MIME)
- Embedding images in HTML/CSS
- API tokens, JWTs
- Anywhere binary data must go through text-only systems

5 Text File Formats

5.1 Why So Many Formats?

Different formats optimize for different goals:

- **Human readability** (Markdown, plain text)
- **Machine parsing** (JSON, XML)
- **Typesetting** (LaTeX, HTML)
- **Interoperability** (CSV, email)

5.2 JSON (JavaScript Object Notation)[6]

Deep dive: Most common data interchange format on the web

Data types:

- String: `"text"` (UTF-8, must use double quotes)
- Number: `42`, `3.14`, `1.2e-10`
- Boolean: `true`, `false`
- Null: `null`
- Array: `[1, 2, 3]`
- Object: `{"key": "value"}`

Key properties:

- Text-based (human-readable)
- Language-agnostic
- Self-describing
- No comments (by design!)
- No binary data (must use base64)

Why use JSON despite overhead?

- No schema needed (self-describing)
- Human-readable
- Easy to debug
- Flexible (schema can evolve)
- Universal parser support

5.3 Other Text Formats

TOML (Tom's Obvious Minimal Language)[7]

- Configuration file format
- More human-friendly than JSON (supports comments!)
- Used by: Rust (Cargo.toml), Python packaging (pyproject.toml)

YAML (YAML Ain't Markup Language)[8]

- Human-readable data serialization
- Indentation-based structure (like Python)
- Used by: Docker (docker-compose.yml), Kubernetes, CI/CD configs

6 Binary Formats

6.1 Why Binary?

Advantages:

- Compact (no text overhead)
- Fast to parse (direct memory mapping)
- Precise (exact bit patterns)

Disadvantages:

- Not human-readable
- Requires exact format knowledge
- Byte-order issues (endianness)

6.2 IEEE 754 Floating-Point Formats[9]

Standard for representing real numbers in computers.

Binary32 (single precision, “float”):

Sign: 1 bit [Exponent: 8 bits][Mantissa: 23 bits]

- Total: 32 bits = 4 bytes
- Range: $\pm 3.4 \times 10^{38}$
- Precision: ~ 7 decimal digits

Binary64 (double precision, “double”):

Sign: 1 bit [Exponent: 11 bits][Mantissa: 52 bits]

- Total: 64 bits = 8 bytes
- Range: $\pm 1.7 \times 10^{308}$
- Precision: ~ 15 decimal digits

Example: The number 1.5

$$1.5_{10} = 1.1_2 = 1 \times 2^0 + 1 \times 2^{-1} \quad (4)$$

Float representation (hex): 0x3FC00000

6.3 Magic Numbers[10, 11]

Definition: Bytes at the start of a file that identify its format (also called “file signatures”)

Examples:

PNG:	89 50 4E 47 0D 0A 1A 0A	(.PNG...)
JPEG:	FF D8 FF	()
GIF:	47 49 46 38	(GIF8)
PDF:	25 50 44 46	(%PDF)
ZIP:	50 4B 03 04	(PK..)
ELF:	7F 45 4C 46	(.ELF)

Purpose:

- Quick file type identification
- Prevents misinterpretation
- Used by `file` command on Unix

7 Information Theory in Engineering Design

7.1 Instruction Set Encoding[12]

Key principle: More frequent operations should use fewer bits (like Huffman coding)

Variable-length encoding in x86:

NOP:	90	(1 byte - very common)
PUSH EAX:	50	(1 byte - common)
MOV EAX,5:	B8 05 00 00 00	(5 bytes - less common)

ARM vs x86 philosophy:

- **ARM (RISC):** Fixed-length (32-bit) instructions
 - Simpler decoding hardware
 - Less code density
 - Predictable performance
- **x86 (CISC):** Variable-length instructions
 - Better code density (smaller programs)
 - Complex decoding (pipeline challenges)
 - Leverages instruction frequency

7.2 Network Protocol Message Formats[13]

Key design pattern: Messages/frames/datagrams have structured formats, typically aligned to 32-bit (4-byte) boundaries.

These formats are like “file formats for network messages” - they define exactly how bits are interpreted.

Why 32-bit alignment?

1. **Hardware efficiency:** Modern CPUs process 32/64-bit words
2. **Simpler parsing:** Can read/write aligned words directly
3. **Padding minimization:** Natural alignment reduces waste
4. **Historical:** 32-bit architectures were standard when protocols designed

Similarities to file formats:

- **Magic numbers** → Protocol version/type fields
- **Length fields** → Total Length, Data Offset
- **Checksums** → Data integrity (like CRC)
- **Variable sections** → Options (like file metadata)

Key differences:

- Network messages are **ephemeral** (not stored long-term)
- **Strict size limits:** MTU constraints (typically 1500 bytes for Ethernet)
- **Performance critical:** Parsed billions of times per second
- **Backward compatibility:** Can't break old implementations

8 Multimedia Formats

8.1 Sound/Audio

8.1.1 Analog to Digital Conversion[14]

1. **Sampling:** Measure amplitude at regular intervals
 - **Sampling rate:** How many samples/second (Hz)
 - **Nyquist-Shannon sampling theorem:** Must sample at $\geq 2 \times$ highest frequency
2. **Quantization:** Round each sample to nearest discrete value
 - **Bit depth:** Bits per sample (8, 16, 24)
 - More bits = less quantization noise

8.1.2 Common Audio Parameters

CD Quality:

- Sample rate: 44,100 Hz (44.1 kHz)
- Bit depth: 16 bits
- Channels: 2 (stereo)
- **Data rate:** $44,100 \times 16 \times 2 = 1,411,200$ bits/sec ≈ 1.4 Mb/s

- **File size:** 1 minute = $1.4 \times 60/8 \approx 10.5$ MB

Voice (telephone):

- Sample rate: 8 kHz
- Bit depth: 8 bits (often with compression)
- Channels: 1 (mono)
- **Data rate:** 64 kb/s

8.1.3 Audio Compression

Lossless: WAV, FLAC, ALAC

- Typical compression: 40-60% of original
- Perfect reconstruction

Lossy: MP3, AAC, Opus[15]

- Uses psychoacoustic models (removes what you can't hear)
- Typical: 128-320 kb/s (10-30 $\times$ compression from CD)

8.2 Images

8.2.1 Digital Image Representation

Raster images:

- Grid of **pixels** (picture elements)
- Each pixel has a color value

Resolution: Width $\times$ Height in pixels

- Example: 1920×1080 (Full HD) = 2,073,600 pixels

8.2.2 Color Spaces

RGB (Red, Green, Blue):

- Each channel: 8 bits (0-255)
- Total: 24 bits/pixel = 16.7 million colors
- Sometimes: 32 bits (24 + 8 alpha for transparency)

8.2.3 Uncompressed Image Size

Example: 1920×1080 **RGB** image

- Pixels: $1,920 \times 1,080 = 2,073,600$
- Bytes per pixel: 3 (RGB)
- **Total:** 6,220,800 bytes $\approx$ 6.2 MB

4K image (3840×2160):

- Uncompressed: $\sim$ 25 MB

8.2.4 Common Formats

PNG (Portable Network Graphics)[16]:

- **Compression:** Lossless (similar to ZIP)
- **Typical ratio:** 2-4 $\times$ for photos, 10-100 $\times$ for graphics
- **Use case:** Graphics, logos, screenshots
- **Features:** Transparency, no quality loss

JPEG (Joint Photographic Experts Group)[17]:

- **Compression:** Lossy (discards high-frequency details)
- **Typical ratio:** 10-50 $\times$ (adjustable quality)
- **Use case:** Photographs
- **Issues:** Artifacts at high compression, no transparency

8.3 Video

8.3.1 Uncompressed Video Data Rate

Example: 1080p @ 30fps

- Resolution: 1920×1080
- Bytes per frame: $1920 \times 1080 \times 3 = 6.2$ MB
- Frames per second: 30
- **Data rate:** $6.2 \text{ MB} \times 30 = 186 \text{ MB/s} = 1.49 \text{ Gb/s}$

4K @ 60fps:

- **Data rate:** $25 \text{ MB} \times 60 = 1.5 \text{ GB/s} = 12 \text{ Gb/s}$

This is completely impractical! Hence compression is essential.

8.3.2 Video Compression Techniques

Intra-frame (spatial):

- Compress each frame like a JPEG
- Typical: 10-20× reduction

Inter-frame (temporal):

- Most frames are similar to previous frames
- Store only the *differences*
- **I-frames:** Full image (keyframes)
- **P-frames:** Predicted from previous frame
- **B-frames:** Predicted from both directions
- Typical: Additional 5-10× reduction

Total compression: 50-200× typical for video

8.3.3 Common Video Codecs[18]

H.264 / AVC:

- Current standard
- Good quality/compression balance
- **Typical rates:** 3-10 Mb/s for 1080p

H.265 / HEVC:

- 25-50% better compression than H.264
- More complex (slower encode/decode)

VP9, AV1:

- Open/royalty-free alternatives
- Similar or better than H.265

8.3.4 Typical Video Data Rates

Note: These are 100-1000× smaller than uncompressed!

Quality	Resolution	Bitrate
YouTube 480p	854×480	0.5-1 Mb/s
YouTube 1080p	1920×1080	3-5 Mb/s
YouTube 4K	3840×2160	13-20 Mb/s
Netflix HD	1920×1080	5-8 Mb/s
Netflix 4K	3840×2160	15-25 Mb/s
Blu-ray	1920×1080	20-40 Mb/s

9 Summary & Key Takeaways

9.1 What We Learned

1. Information is quantifiable

- Entropy $H(X)$ = minimum bits needed
- More randomness = more bits required

2. Units matter

- SI (decimal): 1 kB = 1000 bytes
- Binary: 1 KiB = 1024 bytes
- Network speeds use SI; storage uses binary

3. Data $\neq$ Information

- Same information, many representations
- Tradeoffs: efficiency vs. accessibility

4. Text encoding evolution

- ASCII $\rightarrow$ Unicode (UTF-8, UTF-16)
- Base64 for binary-in-text
- JSON for structured data

5. Binary formats are compact

- IEEE floats for numbers
- Magic numbers identify file types

6. Multimedia requires massive compression

- Audio: 44.1 kHz $\times$ 16-bit = 1.4 Mb/s
- Video: 1080p uncompressed = 1.5 Gb/s
- Compression: 10-200 $\times$ typical

9.2 Connection to Networking

Why this matters for networks:

- **Bandwidth requirements** depend on data representation
 - Streaming 4K video: 15-25 Mb/s (compressed)
 - Without compression: impossible on most networks!
- **Latency sensitivity** varies by data type
 - Voice: < 150 ms tolerable
 - Video conferencing: < 400 ms
 - File transfer: seconds OK
- **Error tolerance** differs
 - Text/data: zero tolerance (use TCP)
 - Audio/video: some loss OK (can use UDP)
- **Protocol design** must handle different data types
 - HTTP for text, JSON, images
 - RTP for real-time audio/video
 - FTP/BitTorrent for large files

9.3 Next Lecture Preview

Physical Layer: Media and Propagation

- How do we actually send bits?
- Copper, fiber, wireless
- Speed of light limitations
- Signal propagation and attenuation

References

- [1] Wikipedia contributors. “Entropy (information theory).” *Wikipedia, The Free Encyclopedia*. [https://en.wikipedia.org/wiki/Entropy_\(information_theory\)](https://en.wikipedia.org/wiki/Entropy_(information_theory))
- [2] Wikipedia contributors. “Binary prefix.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Binary_prefix
- [3] Wikipedia contributors. “ASCII.” *Wikipedia, The Free Encyclopedia*. <https://en.wikipedia.org/wiki/ASCII>
- [4] Wikipedia contributors. “UTF-8.” *Wikipedia, The Free Encyclopedia*. <https://en.wikipedia.org/wiki/UTF-8>

- [5] Wikipedia contributors. “Base64.” *Wikipedia, The Free Encyclopedia*. <https://en.wikipedia.org/wiki/Base64>
- [6] Wikipedia contributors. “JSON.” *Wikipedia, The Free Encyclopedia*. <https://en.wikipedia.org/wiki/JSON>
- [7] Wikipedia contributors. “TOML.” *Wikipedia, The Free Encyclopedia*. <https://en.wikipedia.org/wiki/TOML>
- [8] Wikipedia contributors. “YAML.” *Wikipedia, The Free Encyclopedia*. <https://en.wikipedia.org/wiki/YAML>
- [9] Wikipedia contributors. “IEEE 754.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/IEEE_754
- [10] Wikipedia contributors. “Magic number (programming).” *Wikipedia, The Free Encyclopedia*. [https://en.wikipedia.org/wiki/Magic_number_\(programming\)](https://en.wikipedia.org/wiki/Magic_number_(programming))
- [11] Wikipedia contributors. “List of file signatures.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/List_of_file_signatures
- [12] Wikipedia contributors. “Instruction set architecture.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Instruction_set_architecture
- [13] Wikipedia contributors. “Internet Protocol version 4.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Internet_Protocol_version_4
- [14] Wikipedia contributors. “Nyquist–Shannon sampling theorem.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Nyquist-Shannon_sampling_theorem
- [15] Wikipedia contributors. “Psychoacoustics.” *Wikipedia, The Free Encyclopedia*. <https://en.wikipedia.org/wiki/Psychoacoustics>
- [16] Wikipedia contributors. “PNG.” *Wikipedia, The Free Encyclopedia*. <https://en.wikipedia.org/wiki/PNG>
- [17] Wikipedia contributors. “JPEG.” *Wikipedia, The Free Encyclopedia*. <https://en.wikipedia.org/wiki/JPEG>
- [18] Wikipedia contributors. “Advanced Video Coding.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Advanced_Video_Coding